Budget Tracker

A self-hosted, privacy-first alternative to bank-linked AI budgeting apps

PROBLEM

AI-powered budgeting tools (Copilot Money, Cleo, etc.) get their intelligence by linking directly to bank accounts through an aggregator like Plaid — handing a third party read access to every transaction a user makes. Privacy-conscious people are stuck choosing between “dumb but private” spreadsheets and “smart but exposed” fintech apps.

SOLUTION

Budget Tracker gives users the same AI-level insight — spend analysis, forecasting, proactive nudges — without ever connecting a bank account to a third party. Transactions are entered manually or imported from a CSV the user exported themselves, and the entire app, including its AI layer, runs on infrastructure the user controls.

How AI is used

Five AI features sit on top of a deterministic analytics layer computed straight from the user's own data. The LLM narrates and answers in natural language, but every number it cites is pre-computed and passed in — it is instructed to use only the supplied JSON, never to invent a figure. Every feature has a rule-based / templated fallback, so *LLM_ENABLED=false* degrades quality without breaking functionality.

Conversational Assistant	Free-text spending questions, routed to the relevant analytics endpoints and answered from that data only
Weekly / monthly recaps	Auto-generated income, expense, savings-rate, and net-worth summaries on a background schedule
Forecasting & what-if scenarios	Cash-flow projection from trailing averages, plus scenario modeling from free-text or a form
Subscription & anomaly detection	Flags new recurring charges, price creep, and category outliers vs. the user's own baseline
Behavioral nudging	Rule engine raises budget-pace and goal-pace nudges; LLM phrases the message

The LLM provider is pluggable: a fully local/self-hosted model (Ollama, LM Studio) by default for maximum privacy, or the Claude API when cloud inference is preferred — selected with one config value, same interface either way.

Key learnings

- **Architecture vs. requirements tension is solvable without compromise.** The contest's AI requirement assumes Claude-API-driven features; this project's core thesis is privacy-first, local-only inference. Making the LLM provider pluggable — same interface, config-selected — kept the privacy thesis intact while still supporting real Claude usage.
- **Client-side validation is not validation.** A security pass found a password-length rule enforced only in the React form, trivially bypassed via a direct API call, plus a status endpoint that leaked LLM configuration without requiring login. Both were server-side gaps invisible from the UI.
- **Deterministic-first AI design pays for itself at test time.** Because every AI feature is built on a rules/analytics layer with a non-AI fallback, the full test suite and every demo path run with the LLM off — nothing about correctness depends on model availability.
- **Mobile-responsive is not automatic.** A fixed-width sidebar quietly broke the entire app on phone-sized viewports until a dedicated mobile pass caught it. Desktop-only development hides layout bugs that a real device check surfaces immediately.

Stack FastAPI · SQLAlchemy/Alembic · SQLite · React/TypeScript · Vite · Tailwind **Live demo** 54.175.240.105